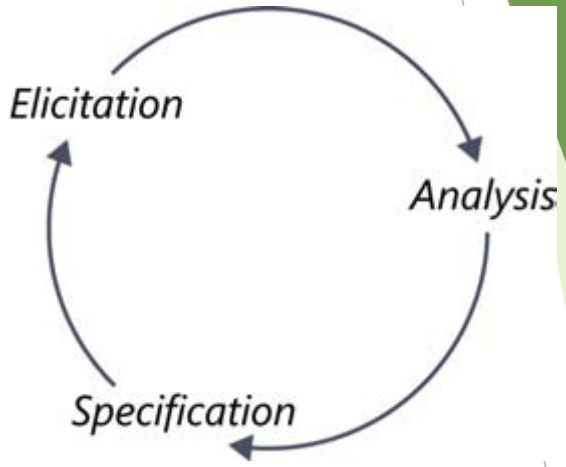
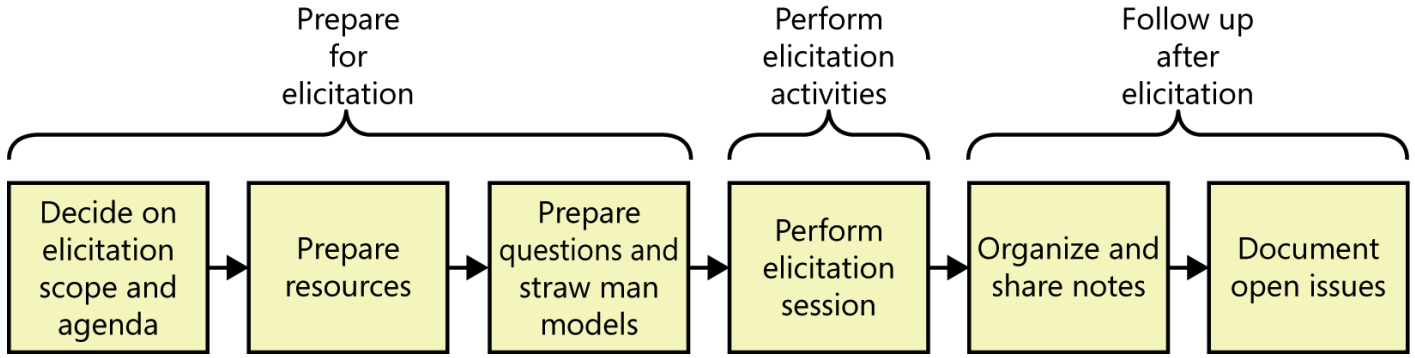
Software Requirements Engineering

Lecture 5: Software Requirements Elicitation

Review

1. Requirements elicitation process
2. Software requirements elicitation techniques





Outline

1. Requirement elicitation techniques
2. Planning elicitation on your project

Requirements elicitation techniques

- Interviews
- Workshops
- Scenarios and use cases
- Focus groups
- Observations
- Questionnaires
- System interface analysis
- User interface analysis
- Document analysis

Stories and scenarios

- Scenarios and user stories are real-life examples of how a system can be used.
- Stories and scenarios are a description of how a system may be used for a particular task.
- Because they are based on a practical situation, stakeholders can relate to them and can comment on their situation with respect to the story.

Scenarios

- Scenarios are structured forms of user story
- Scenarios can be particularly useful for adding detail to an outline requirements description.
- A scenario starts with an outline of the interaction. During the elicitation process, details are added to this to create a complete description of that interaction.
- Scenarios should include
  - A description of the starting situation;
  - A description of the normal flow of events;
  - A description of what can go wrong;
  - Information about other concurrent activities;
  - A description of the state when the scenario finishes.

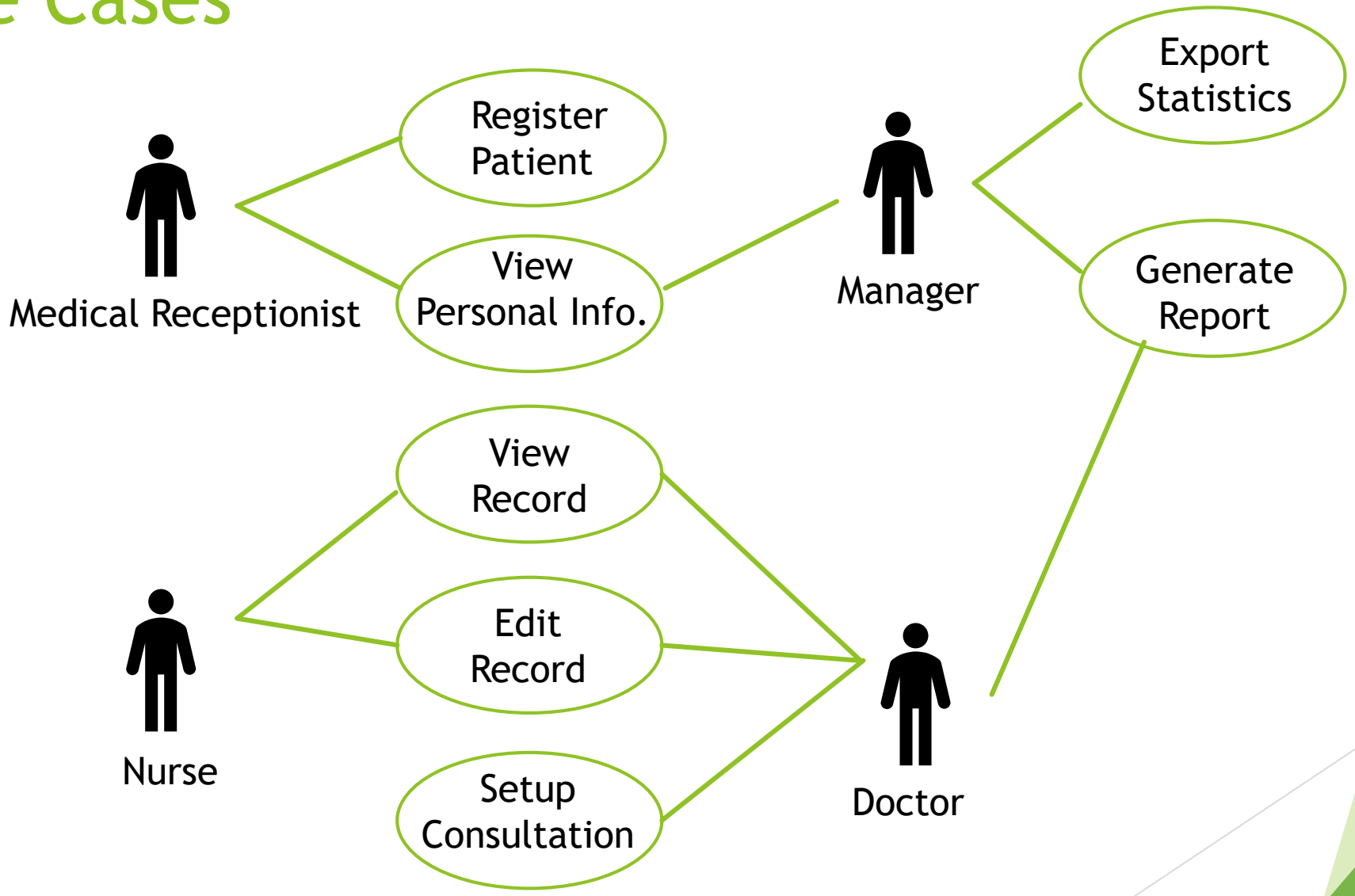
Use Cases

- Use cases are a requirements discovery technique that is a fundamental feature of the unified modeling language.
- In the simplest form, a use case identifies the actors involved in an interaction and names the type of interaction.
- Use cases are documented using a high-level use case diagram.
- The set of use cases represents all of the possible interactions that will be described in the system requirements.

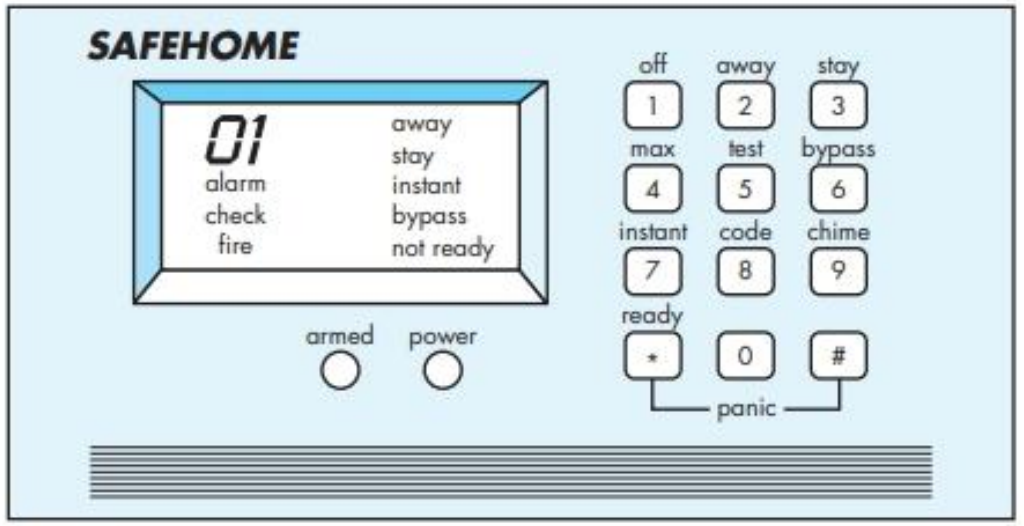
Use Cases

- The first step is defining the set of “actors” that will be involved in the story.
- Actors are the different people (or devices) that use the system or product within the context of the function and behavior that is to be described.
- A typical user may play a number of different roles when using a system.
- An actor represents a class of external entities that play just one role in the context of the use case.
- Because requirements elicitation is an evolutionary activity, not all actors are identified during the first iteration.

Use Cases

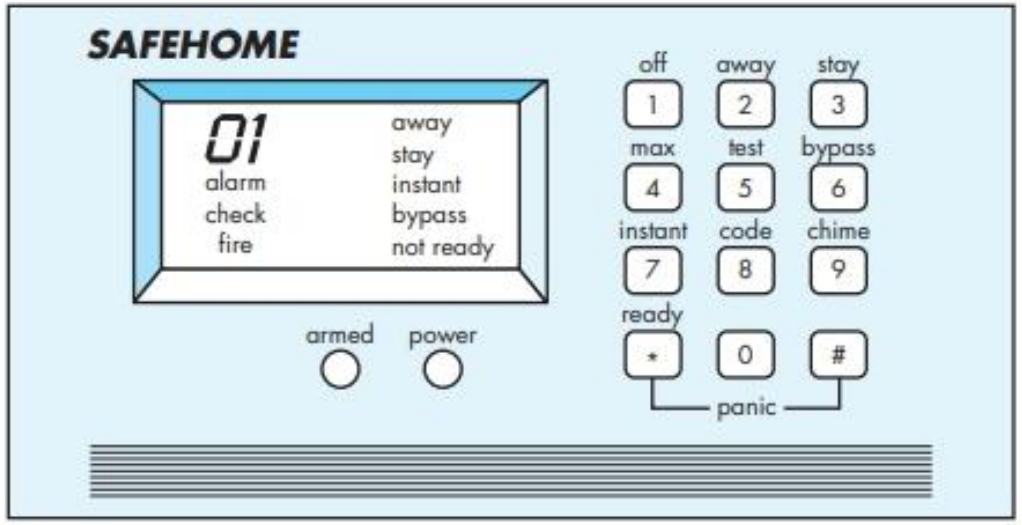

Register Patient
View Personal Info.
Medical Receptionist
Export Statistics
Generate Report
Manager
View Record
Edit Record
Setup Consultation
Nurse
Doctor

Use Cases



- SafeHome Example
- The homeowner actor interacts with the home security function in a number of different ways using either the alarm control panel or a PC.
- The homeowner (1) enters a password to allow all other interactions, (2) inquires about the status of a security zone, (3) inquires about the status of a sensor, (4) presses the panic button in an emergency, and (5) activates/deactivates the security system.

Use Cases



- SafeHome basic use case for system activation:

1. The homeowner observes the SafeHome control panel to determine if the system is ready for input. If the system is not ready, a not ready message is displayed on the LCD display, and the homeowner must physically close windows or doors so that the not ready message disappears.
2. The homeowner uses the keypad to key in a four-digit password. The password is compared with the valid password stored in the system. If the password is incorrect, the control panel will beep once and reset itself for additional input. If the password is correct, the control panel awaits further action.

Use Cases

- SafeHome basic use case for system activation:

3. The homeowner selects and keys in stay or away to activate the system. Stay activates only perimeter sensors (inside motion detecting sensors are deactivated). Away activates all sensors.
4. When activation occurs, a red alarm light can be observed by the homeowner.

Use case:	*InitiateMonitoring*
Primary actor:	Homeowner.
Goal in context:	To set the system to monitor sensors when the homeowner leaves the house or remains inside.
Preconditions:	System has been programmed for a password and to recognize various sensors.
Trigger:	The homeowner decides to "set" the system, that is, to turn on the alarm functions.

Use Cases

Scenario:

1. Homeowner: observes control panel
2. Homeowner: enters password
3. Homeowner: selects "stay" or "away"
4. Homeowner: observes read alarm light to indicate that *SafeHome* has been armed

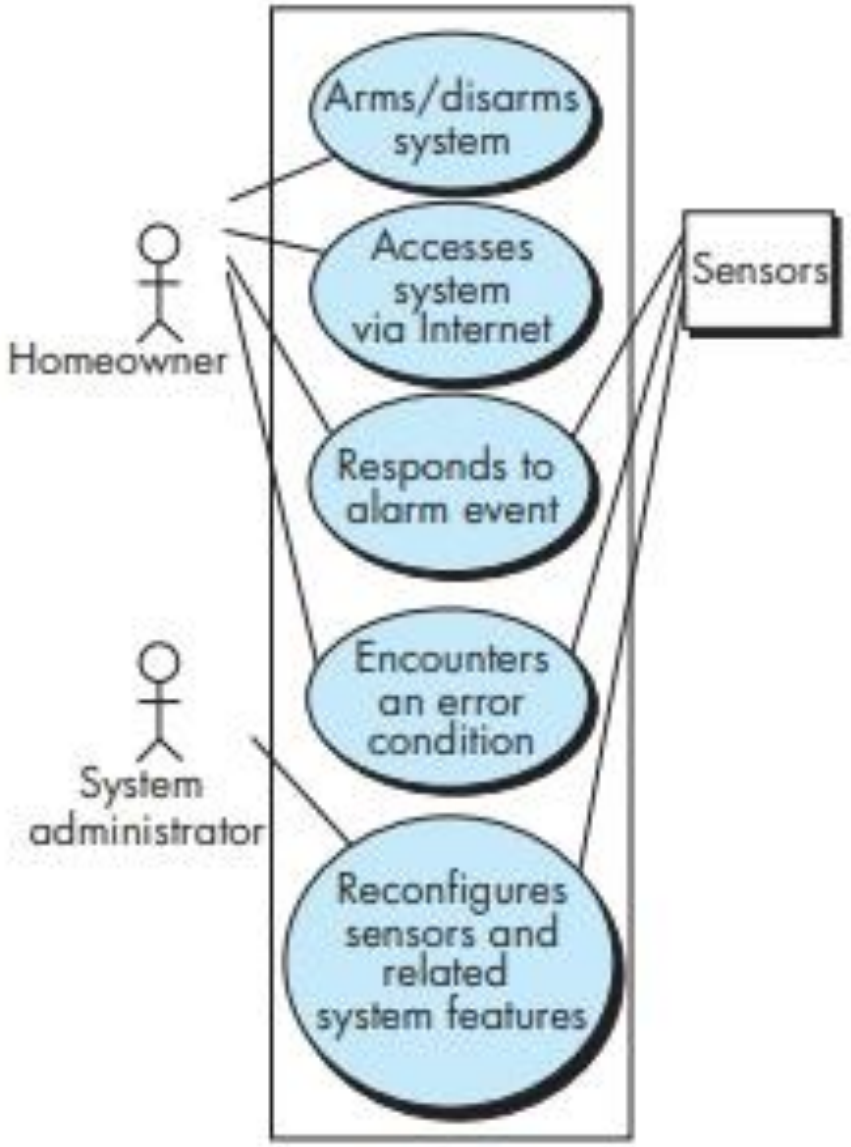
Priority:	Essential, must be implemented
When available:	First increment
Frequency of use:	Many times per day
Channel to actor:	Via control panel interface
Secondary actors:	Support technician, sensors

- Open issues:

1. Should there be a way to activate the system without the use of a password or with an abbreviated password?
2. Should the control panel display additional text messages?
3. How much time does the homeowner have to enter the password from the time the first key is pressed?
4. Is there a way to deactivate the system before it actually activates?

Use Cases

- UML use case diagram for SafeHome home security function



Use Cases

- Scenarios and use cases are effective techniques for eliciting requirements from stakeholders who interact directly with the system.
- Each type of interaction can be represented as a use case.
- They are not as effective for eliciting constraints or high-level business and non-functional requirements or for discovering domain requirements.

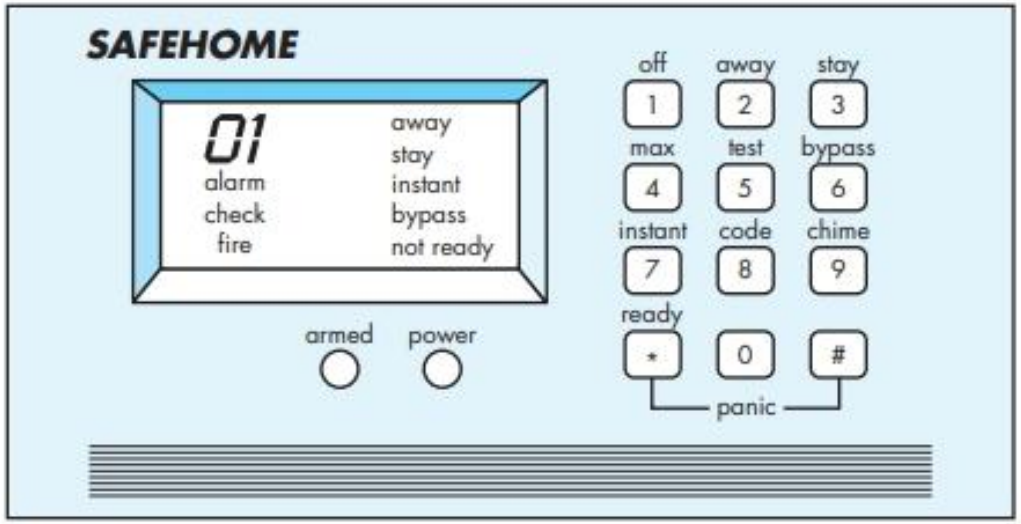
Observations

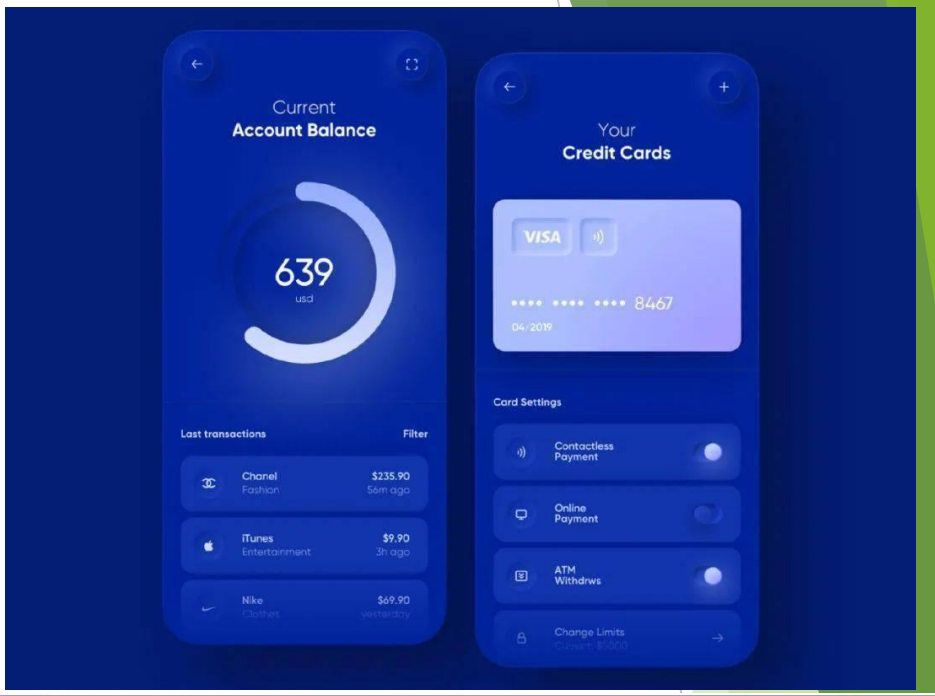
- Observations are time consuming, so they aren't suitable for every user or every task.
- Select important or high-risk tasks and multiple user classes for observations.
- Observations can be silent or interactive.
  - Silent observations are appropriate when busy users cannot be interrupted.
  - Interactive observations allow the BA to interrupt the user mid-task and ask a question.
- Document what you observe for further analysis after the session.

Interface analysis

- Interface analysis is an independent elicitation technique that entails examining the systems to which your system connects.
- System interface analysis reveals functional requirements regarding the exchange of data and services between systems.
- User interface (UI) analysis is an independent elicitation technique in which you study existing systems to discover user and functional requirements.

Interface analysis









Choose the right techniques

	Interviews	Workshops	Focus groups	Observations	Questionnaires	System interface analysis	User interface analysis	Document analysis
Mass-market software	x		x		x			
Internal corporate software	x	x	x	x		x		x
Replacing existing system	x	x		x		x	x	x
Enhancing existing system	x	x				x	x	x
New application	x	x				x		
Packaged software implementation	x	x		x		x		x
Embedded systems	x	x				x		x
Geographically distributed stakeholders	x	x			x			

Good practices

- Define product vision and project scope
  - The vision and scope document contains the product's business requirements.
  - The vision statement gives all stakeholders a common understanding of the product's outcome.
  - The scope defines the boundary between what's in and what's out for a specific release or iteration.
- Identify user classes and their characteristics
  - To avoid overlooking the needs of any user community
  - They might differ in frequency of use, features used, privilege levels, or experience.
- Select a product champion for each user class
  - The product champion presents the needs of the user class and makes decisions on its behalf.

Good practices

- Conduct focus groups with typical users
  - Focus groups are particularly valuable for commercial product development, for which you might have a large and diverse customer base.
- Work with user representatives to identify user requirements
  - User requirements can be expressed in the form of use cases, user stories, or scenarios.
  - Discuss the interactions between the users and the system that will allow them to complete each task.
- Identify system events and responses
  - List the external events that the system can experience and its expected response to each event.
  - Signal events are control signals or data received from external hardware devices.
  - Temporal, or time-based, events trigger a response
  - Business events trigger use cases in business applications.

Good practices

- Hold elicitation interviews and facilitated elicitation workshops
  - Interviews can be performed one-on-one or with a small group of stakeholders.
  - discuss only the specific requirements
- Observe users performing their jobs
  - Watching users perform their business tasks establishes a context for their potential use of a new application.
  - Simple process flow diagrams can depict the steps and decisions involved and show how different user groups interact.
- Distribute questionnaires
  - Questionnaires are a way to survey large groups of users to determine what they need.
  - If questions are well written, questionnaires can help you quickly determine analytical information about needs.

Good practices

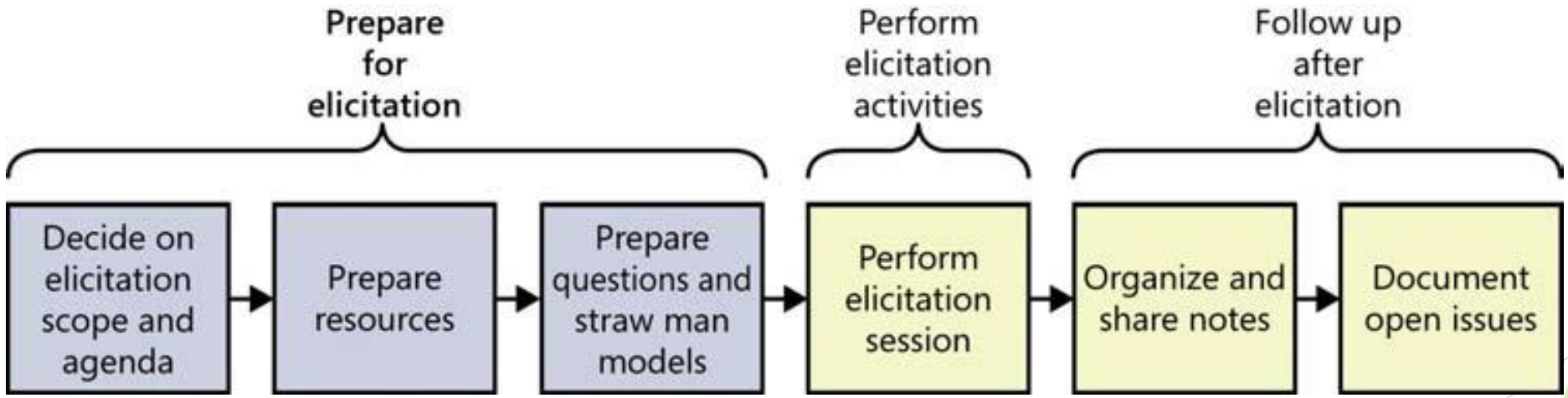
- Perform document analysis
  - Existing documentation can help reveal how systems currently work or what they are supposed to do.
  - Documentation includes any written information about current systems, business processes, requirements specifications, competitor research, and user manuals.
- Examine problem reports of current systems for requirement ideas
  - Help desk and support staff can provide valuable input into the requirements for future development work.
- Reuse existing requirements
  - Reuse or adapt the existing software components.
  - Other good candidates for reuse include glossaries, data models and definitions…

Planning elicitation on your project

- Plan the approach to requirements elicitation early in the project.
- Your plan should address the following items:
  - Elicitation objectives
  - Elicitation strategy and planned techniques
  - Schedule and resource estimates
  - Documents and systems needed for independent elicitation
  - Expected products of elicitation efforts
  - Elicitation risks

Preparing for elicitation activities

Prepare for the elicitation activities in order to make the best use of everyone's time.



Preparing for elicitation activities

Tips:

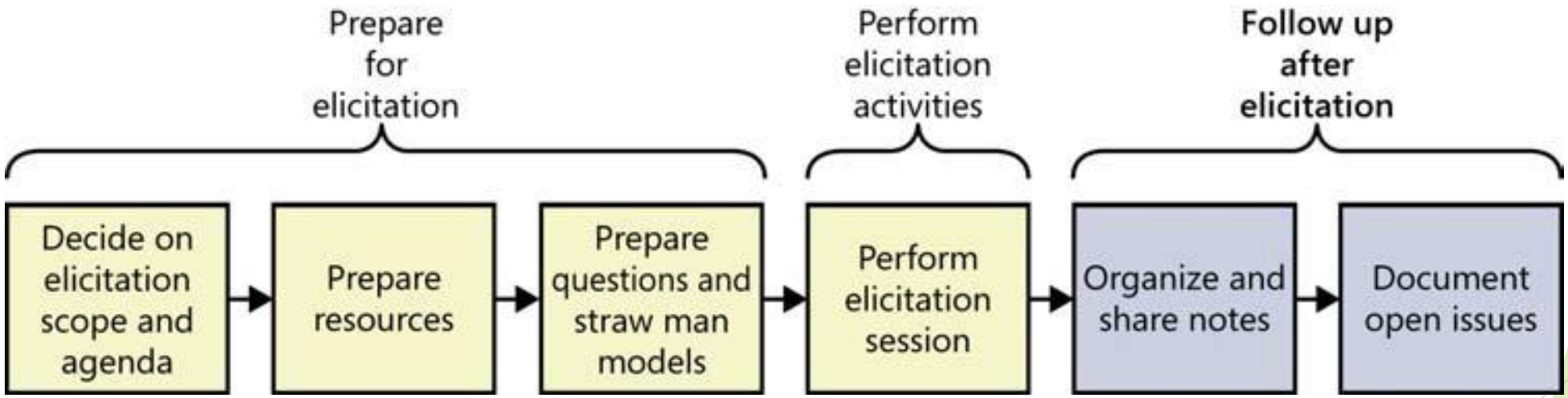
- Plan session scope and agenda
- Prepare resources
- Learn about the stakeholders
- Prepare questions
- Prepare straw man models

Performing elicitation activities

Tips:

- Educate stakeholders
- Take good notes
- Exploit the physical space

Following up after elicitation

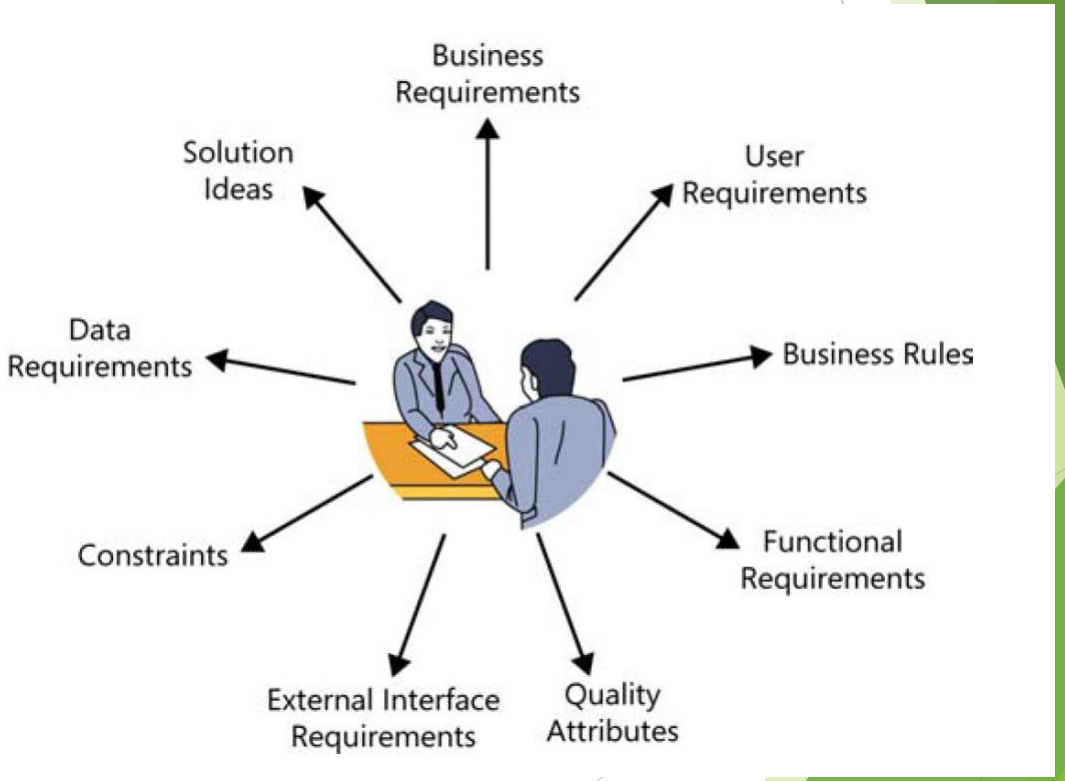


Following up after elicitation

- You need to organize and share your notes, document open issues, and classify the newly gathered information.
- Organizing and sharing the notes
  - Consolidate your input from multiple sources.
  - Review and update your notes soon after the session is complete.
- Documenting open issues
  - items that need to be further explored
  - knowledge gaps you need to close
  - new questions

Classifying customer input

- Don’t expect your customers to present a succinct, complete, and well-organized list of their needs.
- Analysts must classify the requirements information they hear into various categories so that they can document and use it appropriately.
- During elicitation activities, make quick notations in your notes if you detect that some bit of information is one of these types.



Classifying customer input

- As with many categorizations, the information gathered might not fit precisely into these nine buckets.
- Anything that doesn't fit into one of these categories might be:
  - A project requirement not related to the software development, such as the need to train users on the new system.
  - A project constraint, such as a cost or schedule restriction
  - An assumption or a dependency.
  - Additional information of a historical, context-setting, or descriptive nature.
  - Extraneous information that does not add value.
- Elicitation participants won't simply tell you, "Here comes a business requirement."

Some suggestions

- Business requirements
- Anything that describes the financial, marketplace, or other business benefit that either customers or the developing organization wish to gain from the product is a business requirement.
- Listen for statements about the value that buyers or users of the software will receive, such as these:
  - “Increase market share in region X by Y percent within Z months.”
  - “Save $X per year on electricity now wasted by inefficient units.”

Some suggestions

- User requirements
- General statements of user goals or business tasks that users need to perform are user requirements, most typically represented as use cases, scenarios, or user stories.
- A user who says, “I need to <do something>” is probably describing a user requirement, as in the following examples:
  - “I need to print a mailing label for a package.”
  - “As the lead machine operator, I need to calibrate the pump controller first thing every morning.”

Some suggestions

- Business rules
- When a customer says that only certain users can perform an activity under specific conditions, he might be presenting a business rule.
- These aren't software requirements as they stand, but you might derive some functional requirements to enforce the rules.
- Phrases such as "Must comply with . . . ," "If <some condition is true>, then <something happens>," or "Must be calculated according to . . . " suggest that the user is describing a business rule.
- Here are some examples:
  - "A new client must pay 30 percent of the estimated consulting fee and travel expenses in advance."
  - "Time-off approvals must comply with the company's HR vacation policy."

Some suggestions

- Functional requirements
- Functional requirements describe the observable behaviors the system will exhibit under certain conditions and the actions the system will let users take.
- Here are some examples of functional requirements as you might hear them from users:
  - “If the pressure exceeds 40.0 psi, the high-pressure warning light should come on.”
  - “The user must be able to sort the project list in forward and reverse alphabetical order.”
- The BA will need to craft these into more precise specifications.

Some suggestions

- Quality attributes
- Statements that describe how well the system does something are quality attributes.
- Listen for words that describe desirable system characteristics: fast, easy, user-friendly, reliable, secure. You'll need to work with the users to understand just what they mean by these ambiguous and subjective terms so that you can write clear, verifiable quality goals.
- The following examples suggest what quality attributes might sound like when described by users:
  - "The mobile software must respond quickly to touch commands."
  - "The shopping cart mechanism has to be simple to use so my new customers don't abandon the purchase."

Some suggestions

- External interface requirements
- Requirements in this category describe the connections between your system and the rest of the universe
- Phrases such as “Must read signals from . . . ,” “Must send messages to . . . ,” “Must be able to read files in <format>,” and “User interface elements must conform to <a standard>” indicate that the customer is describing an external interface requirement.
- Following are some examples:
  - “The manufacturing execution system must control the wafer sorter.”
  - “The mobile app should send the check image to the bank after I photograph the check I’m depositing.”

Some suggestions

- Constraints
- Design and implementation constraints legitimately restrict the options available to the Developer. Devices with embedded software often must respect physical constraints such as size, weight, and interface connections. Phrases that indicate that the customer is describing a design or implementation constraint include: “Must be written in <a specific programming language>,” “Cannot exceed <some limit>,” and “Must use <a specific user interface control>.”
- The following are examples of constraints that a customer might present:
  - “Files submitted electronically cannot exceed 10 MB in size.”
  - “The browser must use 256-bit encryption for all secure transactions.”
- As with functional requirements, don’t just transcribe the user’s statement of a constraint.
- Ask why the constraint exists, confirm its validity, and record the rationale for including it as a requirement.

Some suggestions

- Data requirements
- Customers are presenting a data requirement whenever they describe the format, data type, allowed values, or default value for a data element; the composition of a complex business data structure; or a report to be generated.
- Some examples of data requirements are as follows:
  - “The ZIP code has five digits, followed by an optional hyphen and four digits that default to 0000.”
  - “An order consists of the customer’s identity, shipping information, and one or more products, each of which includes the product number, number of units, unit price, and total price.”

Some suggestions

- Solution ideas
- Many “requirements” from users are really solution ideas. Someone who describes a specific way to interact with the system to perform some action is suggesting a solution.
- The business analyst needs to probe below the surface of a solution idea to get to the real requirement. Repeatedly asking “why” the user needs it to work this way will likely reveal the true need.
- For instance, passwords are just one of several possible ways to implement a security requirement. Two other examples of solution ideas are the following:
  - “Then I select the state where I want to send the package from a drop-down list.”
  - “The phone has to allow the user to swipe with a finger to navigate between screens.”

Caution!

- Balance stakeholder representation
- Define scope appropriately
- Avoid the requirements-versus-design argument
- Research within reason

You will never document 100 percent of the requirements for a system!

You need to take care of the assumed and implied requirements

Finding missing requirements

- Decompose high-level requirements into enough detail
- Ensure that all user classes have provided input.
- Trace system requirements, user requirements, event-response lists, and business rules to their corresponding functional requirements
- Check boundary values for missing requirements
- Represent requirements information in more than one way.
- Create a checklist of common functional areas
- A data model can reveal missing functionality.
- Sets of requirements with complex Boolean logic (ANDs, ORs, and NOTs) often are incomplete.

Finalize the elicitation process

Perhaps you are done if:

- The users can’t think of any more use cases or user stories.
- New scenarios don’t lead to any new functional requirements.
- Users repeat issues
- New requirements are all deemed to be out of scope or low priority.
- The users are proposing capabilities for the future system.
- Developers and testers who review the requirements for an area raise few questions.

In the next lecture…

- We will talk about requirements analysis.

Question session